

Token-Benchmark: Methodik & Ergebnisse

Vollständiger Beleg — Versuchsaufbau, Methodik, Tabellen, Qualität

Datenquelle: benchmark-9a72151a.json

Datum: Juli 2026

Runs: 540 (n=10 je Kombination, ausser Real-Task n=5)

Harnesses: Pi 0.80.2 | Claude Code 2.1.170 / 2.1.196

Internes Dokument · SelectLine Software AG

1 Kurzzusammenfassung

Dieser POC vergleicht zwei KI-Coding-Harnesses — Pi (minimalistisch) und Claude Code (feature-reich) — bei identischen Prompts und Modellen. Gemessen werden Token-Verbrauch, Kosten und Antwortqualität.

Die drei belastbarsten Befunde:

- Token-Overhead: Claude Code sendet $\sim 8\times$ mehr Tokens pro Anfrage als Pi ($\sim 29\,000$ vs. $\sim 3\,500$ Tokens), unabhängig vom gewählten Modell.
- Kostenoverhead: Wegen günstigem Cache-Read ist der Kosten-Faktor kleiner: $\sim 4\text{--}5\times$ (nicht $8\times$). Tokens zeigen den Umfang, Kosten den finanziellen Effekt.
- Qualität: Kein bedeutsamer Unterschied. Blindtest (27 Paare, Richter Opus 4.8): Pi 3 Siege / CC 13 / 11 Unentschieden. Beide $\bar{O} > 4,3$ auf einer 5-Punkte-Skala.

Robuster Befund: Die Overhead-Kernzahl ($\sim 8\times$ Tokens) basiert ausschließlich auf Single-Turn-Tasks ($n=10$ je Kombination) und ist OS-unabhängig. Tool-Tasks (medium-bash) wurden fair im Container gemessen (beide Harnesses nativ Linux).

2 Versuchsaufbau

Versuchsmatrix: 2 Harnesses $\times$ 3 Modelle $\times$ 9 synthetische Tasks + 1 Real-Task.

- Harnesses: Pi 0.80.2 (minimalistisch) und Claude Code 2.1.170/2.1.196 (feature-reich)
- Modelle: Haiku 4.5, Sonnet 4.6, Opus 4.8 (identische Anthropic-Modelle in beiden Harnesses)
- Wiederholungen: $n=10$ je Kombination (Real-Task $n=5$)
- Sandbox: leeres temporäres Arbeitsverzeichnis, kein Projektkontext
- Kosten: einheitlich aus Tokens via pricing.py berechnet (eine Preistabelle für beide)

Modell-IDs (verifiziert)

Label	Pi --model	Claude Code --model
Haiku 4.5	claude-haiku-4-5	haiku
Sonnet 4.6	claude-sonnet-4-6	sonnet
Opus 4.8	claude-opus-4-8	opus

Preise (Stand Juni 2026)

Alle Kosten einheitlich berechnet aus: $\text{input} \times \text{Preis} + \text{cache_read} \times 0,10 \times \text{Preis} + \text{cache_write} \times 1,25 \times \text{Preis} + \text{output} \times \text{Output-Preis}$.

Modell	Input (\$/MTok)	Output (\$/MTok)	Cache-Read	Cache-Write
Haiku 4.5	\$1,00	\$5,00	\$0,10	\$1,25
Sonnet 4.6	\$3,00	\$15,00	\$0,30	\$3,75
Opus 4.8	\$15,00	\$75,00	\$1,50	\$18,75

3 Methodik

Token-Felder

Jede API-Antwort enthält folgende Token-Felder:

Feld	Bedeutung
input	Nicht-gecachte Eingabe-Tokens (User-Nachricht + ggf. ungecachter System-Prompt)
cache_read	Aus dem Anthropic-Prompt-Cache gelesene Tokens (günstig: $0,10\times$ Input-Preis)
cache_write	In den Cache geschriebene Tokens ($1,25\times$ Input-Preis)

Feld	Bedeutung
output	Vom Modell erzeugte Antwort-Tokens
overhead	KERNKENNZAHLE: input + cache_read + cache_write — gesamter mitgeschickter Kontext

Verwendete Flags

Pi:

```
pi -p --mode json --no-session --no-context-files --no-prompt-templates --thinking off --model <id>
```

Claude Code:

```
claude -p --output-format json --model <id> --allow-dangerously-skip-permissions
```

Overhead-Definition & Baseline-Task

Der Harness-Overhead = input + cache_read + cache_write. Das ist der gesamte pro Anfrage mitgeschickte Kontext ohne die Antwort. Beim Task 'baseline-overhead' (Prompt: 'Reply with exactly: OK') ist dieser Wert praktisch identisch mit dem System-Prompt + Tool-Definitionen — der eigentliche Nutzungsanteil ist minimal.

Pi turn_end-Summierung (multi-turn)

Pi meldet Tokens pro Turn, nicht kumulativ. Bei Tool-Runs werden alle turn_end-Events summiert (ADR-0002). Nur das letzte turn_end zu lesen liefert zu niedrige (falsche) Zahlen.

4 Verifizierte Cache-Semantik

Beobachtet in Referenzlauf dcf6c6db (Rohdaten: docs/evidence/), bestätigt in benchmark-9a72151a.json.

Pi: modellabhängiges Caching (ADR-0001)

Modell	run#0 (kalt)	run#1+ (warm)	Overhead (stabil)
Haiku 4.5	input≈3070, cache=0	input≈3068, cache=0	~3.069
Sonnet 4.6	cache_write≈3066, input=3	cache_read≈1870 + write≈1196	~3.069
Opus 4.8	cache_write≈3784, input=2	cache_read≈2504 + write≈1280	~3.786

Wichtig: Der Token-Overhead ist kalt/warm-stabil. Nur die Kosten variieren (Pi Sonnet kalt ~\$0,012 vs. warm ~\$0,005 — Faktor ~2,3×).

Claude Code: konsistentes Caching

CC nutzt server-seitiges Prompt-Caching konsistent über alle Modelle. Gesamtoverhead ~27-29k Tokens, aufgeteilt in:

- cache_read ≈ 20.900-21.500 Tokens (stabiler Kern, bei run#0 bereits gecacht — möglicherweise Anthropic-seitig persistent für CC als offizielles Tool)
- cache_write ≈ 3.800-7.800 Tokens (session-spezifischer dynamischer Teil)
- input ≈ 10 Tokens (Nachrichten-Boundary)

5 Task-Übersicht (alle Prompts)

Alle 9 synthetischen Tasks laufen in einer leeren Sandbox (tempfile.TemporaryDirectory). Keine Pfad-Abhängigkeiten. Der Real-Task läuft direkt im echten JavaFX-Repo.

Baseline (Overhead-Messung)

[baseline-overhead] use_tools=Nein num_turns=1

Reply with exactly: OK

Trivial (Wissen / Fakten)

[trivial-fact] use_tools=Nein num_turns=1

In which year did the Berlin Wall fall? Answer in one sentence.

[trivial-math] use_tools=Nein num_turns=1

What is 17 x 23? Show your working in two lines.

Simple (Code / Erklären)

[simple-code] use_tools=Nein num_turns=1

Write a TypeScript function called `debounce` that delays invoking a callback until after `waitMs` milliseconds have elapsed since the last call. Include the function signature with generics and a brief JSDoc comment. No imports needed.

[simple-explain] use_tools=Nein num_turns=1

Explain the difference between `Promise.all()` and `Promise.allSettled()` in JavaScript. Use a bullet list with max 4 points.

Medium (Design / Tool-Nutzung)

[medium-design] use_tools=Nein num_turns=1

Design a minimal REST API for a todo-list application. List the endpoints (method + path), the JSON request/response shape for each, and two common error responses. Keep it under 200 words.

[medium-bash] use_tools=Ja num_turns=1

Using your shell tools, gather the following system information and present it as a formatted table:

1. Current working directory and current username
2. Operating system name and version
3. Number of available CPU cores
4. Total and free disk space on the main volume

Use the appropriate shell commands for your environment (Linux/macOS or Windows). End with one sentence summarising the most notable aspect of this environment.

Complex (Analyse / Refactoring)

[complex-refactor] use_tools=Nein num_turns=1

Review the following Python snippet and provide:

1. A list of at least 3 concrete issues (correctness, style, performance).
2. A refactored version that fixes all of them.
3. A one-sentence explanation of the most important change.

```
```python
def get_user_data(ids):
 result = []
 for id in ids:
 r = requests.get("https://api.example.com/users/" + id)
 data = r.json()
 result.append(data)
 return result
```
```

[complex-analysis] use_tools=Nein num_turns=1

You are reviewing a system where a single PostgreSQL database serves both an OLTP web application (thousands of concurrent short transactions) and an OLAP reporting pipeline (long-running analytical queries). Describe three specific problems this causes and propose a concrete architectural solution for each, referencing real tools or patterns (e.g. read replicas, CQRS, materialised views). Keep your answer under 300 words.

Real-Task (echtes JavaFX-Repo, n=5)**[real-javafx-ui-fixes] use_tools=Ja num_turns=1**

You are working on a JavaFX desktop application (Java, Maven project).

Find and fix the following 4 UI bugs. The full source code is in your working directory.

Use your tools to locate the relevant files, read them, and edit them directly.

After all changes, output a short summary of what you changed in each file.

Bug 1: The connection settings window (Verbindungsinformationen) is not resizable. Make it resizable.

Bug 2: The back button ("Zurück") in the restore detail view has no padding. Add appropriate padding (e.g. 8px on all sides).

Bug 3: In the time field component used in job settings, hours, minutes and seconds are not always formatted with two digits (e.g. "9" instead of "09") when the timeProperty value changes. Fix the formatting for all three fields.

Bug 4: The job overview list has no fixed column widths, so the layout shifts depending on content. Add fixed preferred widths to the main content sections (info, date, duration, stats) so the layout looks table-like and stable.

6 Ergebnisse: Overhead-Kernzahl (Baseline)

Task: baseline-overhead · n=10 je Kombination · Datenquelle: benchmark-9a72151a.json. Overhead = input + cache_read + cache_write.

| Modell | Pi Tok. (med) | CC Tok. (med) | Fakt.Tok | Pi \$ (med) | CC \$ (med) | Fakt.\$ |
|------------|---------------|---------------|-------------|-------------|-------------|-------------|
| Haiku 4.5 | 3,453 | 29,075 | 8.4x | \$0.0035 | \$0.0119 | 3.4x |
| Sonnet 4.6 | 3,453 | 28,250 | 8.2x | \$0.0066 | \$0.0340 | 5.2x |
| Opus 4.8 | 4,319 | 26,955 | 6.2x | \$0.0381 | \$0.1359 | 3.6x |

Interpretation: CC schickt ~8x mehr Overhead-Tokens pro Anfrage als Pi — unabhängig vom Modell. Der Kosten-Faktor ist kleiner (~4-5x), weil der Großteil des CC-Overheads aus günstigem cache_read besteht (0,10x Input-Preis).

7 Ergebnisse: Token-Overhead — alle Tasks

Median-Overhead (Tokens) je Task × Modell × Harness. Synthetische Tasks n=10, Real-Task n=5 (mit Vorbehalt, siehe Abschnitt 11).

| Task | Kompl. | Modell | Pi (Tok) | CC (Tok) | Faktor |
|----------|----------|------------|----------|----------|-------------|
| Baseline | baseline | Haiku 4.5 | 3,453 | 29,075 | 8.4x |
| | | Sonnet 4.6 | 3,453 | 28,250 | 8.2x |
| | | Opus 4.8 | 4,319 | 26,955 | 6.2x |
| Fakt | trivial | Haiku 4.5 | 3,462 | 29,086 | 8.4x |
| | | Sonnet 4.6 | 3,463 | 28,259 | 8.2x |

| Task | Kompl. | Modell | Pi (Tok) | CC (Tok) | Faktor |
|----------|---------|------------|----------|----------|-------------|
| | | Opus 4.8 | 4,334 | 26,971 | 6.2x |
| Mathe | trivial | Haiku 4.5 | 3,464 | 29,088 | 8.4x |
| | | Sonnet 4.6 | 3,465 | 28,261 | 8.2x |
| | | Opus 4.8 | 4,327 | 26,965 | 6.2x |
| Code | simple | Haiku 4.5 | 7,318 | 59,121 | 8.1x |
| | | Sonnet 4.6 | 7,416 | 28,295 | 3.8x |
| | | Opus 4.8 | 4,383 | 27,022 | 6.2x |
| Erklären | simple | Haiku 4.5 | 3,482 | 29,106 | 8.4x |
| | | Sonnet 4.6 | 3,483 | 28,279 | 8.1x |
| | | Opus 4.8 | 4,357 | 26,995 | 6.2x |
| Design | medium | Haiku 4.5 | 5,446 | 29,115 | 5.3x |
| | | Sonnet 4.6 | 3,492 | 28,287 | 8.1x |
| | | Opus 4.8 | 4,366 | 27,004 | 6.2x |
| Bash* | medium | Haiku 4.5 | 7,317 | 51,595 | 7.1x |
| | | Sonnet 4.6 | 7,280 | 49,914 | 6.9x |
| | | Opus 4.8 | 9,072 | 44,298 | 4.9x |
| Refactor | complex | Haiku 4.5 | 8,298 | 29,197 | 3.5x |
| | | Sonnet 4.6 | 3,574 | 28,370 | 7.9x |
| | | Opus 4.8 | 4,454 | 27,091 | 6.1x |
| Analyse | complex | Haiku 4.5 | 3,537 | 29,161 | 8.2x |
| | | Sonnet 4.6 | 3,538 | 28,334 | 8.0x |
| | | Opus 4.8 | 4,443 | 27,081 | 6.1x |

* medium-bash: im Docker-Container fair gemessen (beide Harnesses nativ Linux, ADR-0005).

8 Ergebnisse: Kosten — alle Tasks

Median-Kosten (USD) je Task × Modell × Harness. Kosten einheitlich aus Tokens berechnet (pricing.py).

| Task | Kompl. | Modell | Pi (\$) | CC (\$) | Faktor |
|----------|----------|------------|-----------|-----------|-------------|
| Baseline | baseline | Haiku 4.5 | \$0.00347 | \$0.01190 | 3.4x |
| | | Sonnet 4.6 | \$0.00655 | \$0.03402 | 5.2x |
| | | Opus 4.8 | \$0.03808 | \$0.13586 | 3.6x |
| Fakt | trivial | Haiku 4.5 | \$0.00363 | \$0.01216 | 3.4x |
| | | Sonnet 4.6 | \$0.00671 | \$0.03418 | 5.1x |
| | | Opus 4.8 | \$0.03934 | \$0.13714 | 3.5x |
| Mathe | trivial | Haiku 4.5 | \$0.00373 | \$0.01301 | 3.5x |
| | | Sonnet 4.6 | \$0.00728 | \$0.03458 | 4.7x |
| | | Opus 4.8 | \$0.04065 | \$0.14173 | 3.5x |
| Code | simple | Haiku 4.5 | \$0.00947 | \$0.02117 | 2.2x |
| | | Sonnet 4.6 | \$0.02102 | \$0.03803 | 1.8x |
| | | Opus 4.8 | \$0.07789 | \$0.18733 | 2.4x |

| Task | Kompl. | Modell | Pi (\$) | CC (\$) | Faktor |
|----------|---------|------------|-----------|-----------|-------------|
| Erklären | simple | Haiku 4.5 | \$0.00450 | \$0.01374 | 3.1x |
| | | Sonnet 4.6 | \$0.00969 | \$0.03797 | 3.9x |
| | | Opus 4.8 | \$0.05778 | \$0.16127 | 2.8x |
| Design | medium | Haiku 4.5 | \$0.00789 | \$0.01506 | 1.9x |
| | | Sonnet 4.6 | \$0.01207 | \$0.04085 | 3.4x |
| | | Opus 4.8 | \$0.06881 | \$0.17512 | 2.5x |
| Bash* | medium | Haiku 4.5 | \$0.00939 | \$0.02456 | 2.6x |
| | | Sonnet 4.6 | \$0.01598 | \$0.06836 | 4.3x |
| | | Opus 4.8 | \$0.08640 | \$0.25483 | 2.9x |
| Refactor | complex | Haiku 4.5 | \$0.01637 | \$0.01893 | 1.2x |
| | | Sonnet 4.6 | \$0.01826 | \$0.04959 | 2.7x |
| | | Opus 4.8 | \$0.10242 | \$0.21617 | 2.1x |
| Analyse | complex | Haiku 4.5 | \$0.00630 | \$0.01665 | 2.6x |
| | | Sonnet 4.6 | \$0.01467 | \$0.04250 | 2.9x |
| | | Opus 4.8 | \$0.09827 | \$0.19924 | 2.0x |

Der Kosten-Faktor liegt typisch bei 4-6x für einfache Tasks, kann aber je nach Token-Zusammensetzung (Output-Anteil) variieren.

9 Qualitätsbewertung (Blindtest)

Methodik Blindtest

- Richter: Opus 4.8 (Fallback Sonnet 4.6 bei Opus-Aufgaben)
- Methode: blind, A/B randomisiert, Swap-Test (2 Durchläufe je Paar)
- Kriterien: korrektheit, vollstaendigkeit, anweisungstreue, klarheit, praegnanz (je 1-5 Punkte)
- Paare bewertet: 27
- Fehler: 0
- Position-Bias-Fälle (ausgewertet): 3

Gesamtergebnis

| | Pi | Claude Code | Unentschieden |
|---------------|------|-------------|---------------|
| Siege | 3 | 13 | 11 |
| Ø-Score (1-5) | 4.39 | 4.80 | Delta: 0.41 |

Fazit: Kein bedeutsamer Qualitätsunterschied. Beide Harnesses liefern hohe Qualität (Ø > 4,3/5). CC liegt minimal vorn (+0,41 Punkte), aber auf gleichem Modell und gleichen Prompts. Qualitätsdifferenz ist deutlich kleiner als Overhead-Differenz.

Einzelergebnisse je Task x Modell

| Task | Modell | Erg. | Pi | CC | Begründung |
|-------------------|------------|------------|-----|-----|---|
| baseline-overhead | Haiku 4.5 | Tie | 5.0 | 5.0 | Beide Antworten geben exakt 'OK' aus und erfüllen die Vorgabe identisch perfekt. |
| baseline-overhead | Sonnet 4.6 | Tie | 5.0 | 5.0 | Beide Antworten lauten exakt 'OK' und erfüllen die Vorgabe identisch und perfekt. |

| Task | Modell | Erg. | Pi | CC | Begründung |
|-------------------|------------|------------|-----|-----|--|
| baseline-overhead | Opus 4.8 | Tie | 5.0 | 5.0 | Beide Antworten sind identisch und erfüllen die Aufgabe perfekt. |
| trivial-fact | Haiku 4.5 | CC | 4.7 | 5.0 | Beide sind faktisch korrekt und einsätzig, aber B ist prägnanter ohne überflüssige Zusatzinformation. |
| trivial-fact | Sonnet 4.6 | Tie | 5.0 | 5.0 | Beide Antworten sind faktisch korrekt, vollständig und in einem Satz formuliert; der einzige Unterschied ist die optionale Fettmarkierung, die inhaltlich keinen Unterschied macht. |
| trivial-fact | Opus 4.8 | Tie | 5.0 | 5.0 | Beide Antworten sind identisch – faktisch korrekt, vollständig, prägnant und exakt im geforderten Format. |
| trivial-math | Haiku 4.5 | CC | 4.2 | 5.0 | Beide Antworten sind korrekt, aber B hält die Vorgabe von genau zwei Zeilen ein, während A vier Zeilen Rechenweg verwendet. |
| trivial-math | Sonnet 4.6 | CC | 4.6 | 5.0 | Beide sind korrekt (391), aber B hält die Vorgabe 'zwei Zeilen' exakt ein und ist präziser, während A mehr Zeilen und Zusatzformatierung verwendet. |
| trivial-math | Opus 4.8 | Tie | 5.0 | 5.0 | Beide Antworten sind inhaltlich identisch korrekt, vollständig, auf genau zwei Zeilen und gleich klar strukturiert – lediglich die Darstellungsform unterscheidet sich marginal. |
| simple-code | Haiku 4.5 | Tie | 2.4 | 2.6 | Beide liefern nur Beschreibungen statt des geforderten Codes, doch B ist prägnanter und ebenso vollstaendig. |
| simple-code | Sonnet 4.6 | CC | 2.4 | 5.0 | B liefert die geforderte, lauffähige Funktion mit Signatur und JSDoc, während A nur Erklärungen ohne tatsächliche Implementierung zeigt. |
| simple-code | Opus 4.8 | Tie | 3.8 | 4.1 | Antwort B erklärt technisch präziser (ReturnType<typeof setTimeout>, Browser/Node-Kompatibilität) und zeigt den tatsächlichen Code implizit vollständiger, während A keinen sichtbaren Code liefert und nur auf eine Datei verweist. |
| simple-explain | Haiku 4.5 | CC | 4.6 | 5.0 | Beide sind fachlich korrekt und vollständig, doch B hält das geforderte reine Bullet-Format ohne überflüssige Überschrift exakt ein, während A eine zusätzliche Markdown-Überschrift hinzufügt. |

| Task | Modell | Erg. | Pi | CC | Begründung |
|------------------|------------|------------|-----|-----|--|
| simple-explain | Sonnet 4.6 | CC | 4.7 | 5.0 | Beide sind inhaltlich korrekt und vollstaendig, doch B ist etwas praegnanter ohne Substanzverlust. |
| simple-explain | Opus 4.8 | Pi | 5.0 | 4.7 | Beide Antworten sind fachlich korrekt und vollständig, aber A hält die Vorgaben strenger ein (keine einleitende Prosazeile, kein Fülltext), während B mit 'Here's the difference:' einen unnötigen Satz voranstellt und insgesamt etwas weitschweifiger ist. |
| medium-design | Haiku 4.5 | CC | 4.6 | 5.0 | Beide sind korrekt und vollständig, aber B ist kompakter und liegt sicherer unter dem 200-Wörter-Limit, während A etwas ausführlicher ist. |
| medium-design | Sonnet 4.6 | Tie | 4.6 | 4.8 | B ist vollständiger (GET-by-id), aber A hält das Wortlimit klar ein und ist prägnanter, während B durch Tabelle plus Shapes redundant wirkt. |
| medium-design | Opus 4.8 | Pi | 5.0 | 4.2 | Antwort A ist kompakter und klarer strukturiert – jeder Endpoint zeigt sofort Request und Response, ohne Redundanz; Antwort B verschlechtert die Prägnanz durch doppelte Darstellung mancher Endpoints und einen unnötigen Meta-Kommentar am Ende. |
| complex-refactor | Haiku 4.5 | CC | 2.6 | 4.8 | B liefert vollständigen, korrekten Code für beide Varianten, während A den entscheidenden refaktorierten Code nur referenziert ('see file above') und damit den geforderten Teil 2 nicht erfüllt. |
| complex-refactor | Sonnet 4.6 | CC | 3.7 | 5.0 | Antwort B ist fehlerfrei und prägnant, während Antwort A einen gravierenden Code-Bug enthält (executor wird vor der with-Anweisung verwendet und der futures-Block ist doppelt/falsch verschachtelt). |
| complex-refactor | Opus 4.8 | Pi | 5.0 | 4.8 | Beide Antworten sind inhaltlich stark und vollständig; A ist minimal prägnanter und liefert sauberere URL-Kodierung mit quote(), während B durch Retry-Logik und try/except praktisch wertvoller ist, aber dadurch etwas gesprächiger wirkt. |

| Task | Modell | Erg. | Pi | CC | Begründung |
|------------------|------------|------------|-----|-----|--|
| complex-analysis | Haiku 4.5 | CC | 3.1 | 4.6 | B liefert drei konkret beschriebene Probleme samt passenden Lösungen mit echten Tools und hält das Format ein, während A nur eine vage Zusammenfassung gibt und auf eine nicht vorhandene Datei verweist. |
| complex-analysis | Sonnet 4.6 | CC | 4.5 | 4.9 | Beide sind fachlich korrekt und vollständig, aber B ist präziser, nennt konkrete Patterns wie Transactional Outbox und quantifizierte Auswirkungen, während A durch Tabelle und Trenner mehr Volumen ohne Mehrwert hat. |
| complex-analysis | Opus 4.8 | Tie | 5.0 | 5.0 | Beide Antworten sind inhaltlich identisch korrekt, vollständig und gleich gut strukturiert – Unterschiede beschränken sich auf marginale Reihenfolge- und Formulierungsvarianten ohne sachlichen Mehrwert. |
| medium-bash | Haiku 4.5 | CC | 4.4 | 5.0 | Beide sind faktisch korrekt und vollständig, aber B hält sich präziser an die Ein-Satz-Vorgabe ohne Füllfloskeln, während A einen überladenen Zusammenfassungssatz und einleitendes Geschwafel enthält. |
| medium-bash | Sonnet 4.6 | CC | 4.5 | 5.0 | Beide sind faktisch korrekt und vollständig, aber B ist mit der schlankeren Tabelle prägnanter und klarer strukturiert. |
| medium-bash | Opus 4.8 | Tie | 5.0 | 5.0 | Beide Antworten liefern identische Fakten, vollständige Tabellen und prägnante Zusammenfassungen – B ergänzt nur den Overlay-Dateisystemhinweis, der minimal informativer ist, aber keinen klaren Gesamtvorteil schafft. |

10 Architecture Decision Records (ADRs)

Alle harten Entscheidungen und Befunde mit Datum.

ADR-0001 (2026-06-25): Pi-Caching ist modellabhängig

Pi nutzt je nach Modell unterschiedliches Caching: Haiku 4.5 gar kein Caching (input only), Sonnet 4.6 + Opus 4.8 aktivieren server-seitiges Prompt-Caching ab run#0. Token-Overhead ist kalt/warm-stabil; Kosten variieren (~2,3× kalt/warm).

ADR-0002 (2026-06-25): Pi turn_end-Summierung bei Tool-Runs

Pi meldet Tokens pro Turn, nicht kumulativ. Alle turn_end-Events müssen summiert werden. Das input-Feld ist bei Tool-Runs ~1–3 Tokens (Pi packt den User-Prompt in cacheWrite). total_tokens ist die verlässliche Vergleichsgröße.

ADR-0003 (2026-06-29): medium-bash war environment-asymmetrisch — Aufgabe ersetzt

Ursprünglicher Prompt fragte nach `/usr/share/doc` (Linux-Pfad). CC lief als Windows-Prozess und fand diesen Pfad nicht zuverlässig. Ersetzt durch environment-agnostischen Systeminfo-Task (OS, CPU, RAM, Docker-Version).

ADR-0004 (2026-06-30): Plattform-Asymmetrie CC=Windows, Pi=Linux

claude ist in der WSL-Umgebung ein Windows-Binary (`/mnt/c/.../npm/claude`), pi läuft nativ unter Linux/WSL. Betrifft alle Tool-/Multi-Turn-Tasks (verfälschte Tokens, Turns, Laufzeit). Die Single-Turn-Overhead-Kernzahl ($\sim 8\times$) ist OS-unabhängig und bleibt gültig. Lösung: Docker-Container (beide Harnesses nativ Linux).

ADR-0005 (2026-06-30): Datenquellen-Merge — faire Gesamtsuite

benchmark-d1b7ef63.json (540 Runs, CC=Windows) wurde mit dem fairen medium-bash-Lauf aus dem Container (benchmark-c066a92f.json, CC+Pi=Linux) zur Gesamtsuite benchmark-9a72151a.json zusammengeführt. Gemischte Provenienz ist im JSON dokumentiert. Tool-Faktor medium-bash: 4,7-6,8 $\times$ (statt 14-22 $\times$ Messartefakt).

11 Einschränkungen & offene Fragen

- Cache-TTL für Claude-4-Modelle nicht explizit verifiziert. Annahme: 5 Minuten (analog claude-3). Nicht aus Messdaten direkt ableitbar.
- Stabiler CC-Kern (`cache_read` ~ 21.500 Tokens bereits bei `run#0`) ist Beobachtung, kein belegter Fakt. Mögliche Erklärung: Anthropic-seitiges Langzeit-Caching für CC.
- Pi-Caching-Ursache bei Sonnet/Opus unklar (minimale Schwellwert-Differenz, oder modellspezifische Anthropic-Konfiguration).
- Qualitätsbewertung: nur 27 Paare (baseline/trivial/simple/medium/complex, kein Real-Task). Statistisch begrenzt. Richter = Opus $\rightarrow$ Opus-Ergebnisse mit Sonnet bewertet.
- Modellpreise Stand Juni 2026. Bei Änderungen `pricing.py` aktualisieren.

12 Provenienz

Suite-ID: benchmark-9a72151a

Plattform: Linux-5.15.133.1-microsoft-standard-WSL2-x86_64-with-glibc2.35

Python: 3.10.12

Tool-Versionen: Pi 0.80.2 | Claude Code 2.1.170 (Claude Code)

Wiederholungen: 10

Sandbox: leeres temporaeres Arbeitsverzeichnis pro Lauf (kein Projektkontext)

Merge-Provenienz (ADR-0005)

Basis-Suite: d1b7ef63 — Plattform: Linux-5.15.133.1-microsoft-standard-WSL2-x86_64-with-glibc2.

Override-Suite: c066a92f (Task: medium-bash)

Warnung: GEMISCHTE PROVENIENZ: Single-Turn-Tasks stammen aus dem Basis-Lauf (Claude Code Windows-Binary), 'medium-bash' aus dem Container-Lauf (Claude Code Linux). Der Overhead pro Anfrage ist fuer Single-Turn-Tasks OS-unabhaengig und damit vergleichbar; der Claude-Code-Versionsunterschied ist hier dokumentiert.